NoSQL Verilerinden İlişkisel SQL Şemalarına Dinamik Dönüşüm Motoru Geliştirilmesi

Hasan Hüseyin Yüksel – HANI ASHRAFI

Bilgisayar Mühendisliği Bölümü Kocaeli Üniversitesi Kocaeli, Türkiye

yukselhuseyin58@gmail.com-haofficial122@gmail.com

***ÖZET*: Bu proje, yapısı ve derinliği önceden bilinmeyen, şemasız (schema-less) NoSQL (JSON) verilerini dinamik olarak analiz edip, otomatik normalizasyon kurallarını uygulayarak ilişkisel bir SQL (SQLite) veritabanına dönüştüren jenerik bir motorun geliştirilmesini sunmaktadır. Sistem; iç içe geçmiş nesneleri düzleştirme (flattening), liste (array) yapılarını 1NF kuralları gereği yeni alt tablolara bölme ve bu tablolar arasında dinamik Yabancı Anahtar (Foreign Key) ilişkileri kurma işlemlerini otonom olarak gerçekleştirir. Geliştirilen C# WinForms uygulaması ile veritabanı şemalarının çalışma zamanında (runtime) "hardcoded" olmadan nasıl üretilebileceği kanıtlanmış ve sistemin hata toleransı (kilitlenme ve çakışma önleme) başarılı bir şekilde test edilmiştir.**

I.GİRİŞ

Günümüz yazılım mimarilerinde esnek, şemasız (schema-less) veri yapıları sunan NoSQL formatları (özellikle JSON) ile katı veri bütünlüğü sağlayan ilişkisel veritabanı yönetim sistemleri (RDBMS) arasında büyük bir veri transferi (ETL - Extract, Transform, Load) problemi yaşanmaktadır. "Impedance mismatch" olarak da bilinen bu uyumsuzluk, ağaç (tree) tabanlı ve iç içe geçmiş nesnelerin, iki boyutlu düz tablolara (satır ve sütunlara) aktarılırken veri kaybı yaşanmasına veya normalizasyon kurallarının ihlal edilmesine yol açar. Bu projenin temel motivasyonu, yapısı, sütun adları ve derinliği önceden tanımlanmamış (hardcoded olmayan) herhangi bir karmaşık JSON dosyasını çalışma zamanında (runtime) okuyabilen otonom bir dönüştürücü motoru (Converter Engine) C# ortamında geliştirmektir.

II.YÖNTEM VE ALGORİTMA TASARIMI

Geliştirilen sistem, veritabanı tablolarını ve ilişkileri statik olarak belirlemek yerine, gelen JSON verisinin türüne göre dinamik kararlar alan 4 aşamalı bir "Pipeline" mimarisiyle tasarlanmıştır.

A. Veri Okuma ve Tip Çıkarımı (Type Inference)
JSON dosyası Newtonsoft.Json kütüphanesi ile belleğe JToken ağacı olarak yüklenir. Gelen her anahtar-değer (Key-Value) ikilisi taranır. Sistem; sayıları INTEGER veya REAL, boolean değerleri INTEGER (0 ve 1), tarih formatlarını DATETIME ve metinleri TEXT olarak otonom şekilde SQL veri tiplerine çevirir.

B. Düzleştirme (Flattening)

Veritabanında gereksiz tablo karmaşasını önlemek için MAX_FLATTEN_DEPTH (Maksimum Düzleştirme Derinliği) algoritması kullanılmıştır. İç içe geçmiş objelerde ({ }), ebeveyn ve çocuk anahtarlar birleştirilerek (Örn: adres_sehir) tek bir satıra indirgenir.

C. Liste Yönetimi ve Normalizasyon (1NF, 2NF)
Eğer ayrıştırıcı motor bir dizi/liste ([ ]) ile karşılaşırsa, 1. Normal Form (Atomik Veri) kuralı gereği bu verileri virgülle ayırıp ana tabloya gömmez. Diziyi fark ettiği anda işlemi durdurur, bu dizi için ebeveyn tablonun adından türetilen (Örn: Main_Table_skills) yeni bir alt tablo oluşturur. 2. Normal Form kuralı gereği bu alt tabloya Parent_Id ve Local_Id adında kompozit birincil anahtar atanır ve ana tablonun ID'si Yabancı Anahtar (Foreign Key) olarak otonom şekilde bağlanır.

D. Yalancı Kod (Pseudocode)

Aşağıdaki yalancı kod, sistemin dinamik hiyerarşi çözümleme mantığını özetlemektedir:

```
Fonksiyon NesneyiCozumle (obje, ebeveynTablo,
ebeveynId, derinlik):
    EĞER derinlik > MAX_DERINLIK İSE
        Referans Tablosu (Lookup) oluştur ve ID
döndür.
    hedef_stir = Yeni DinamikTabloSatiri()
    hedef_stir.TabloAdi = ebeveynTablo


    DÖNGÜ: objenin içindeki her (anahtar, deger)
için:
        EĞER deger tipi == Obje İSE:
```

yeni_onEk = anahtar + "_" // Flattening (Düzleştirme)

NesneyiCozumle(deger, ebeveynTablo, hedef_stir.Id, derinlik + 1)

DEĞİLSE EĞER deger tipi == Dizi İSE:

alt_tablo_adi = ebeveynTablo + "_" + anahtar

DÖNGÜ: dizideki her eleman için:

BagimliSatirOlustur(eleman, alt_tablo_adi, hedef_stir.Id)

DEĞİLSE: // İlkel Değerler (String, Int, Boolean)

EĞER anahtar == "id" İSE anahtar = "json_id" // Çakışma Önle

SQL_Tipi = VeriTipiniTespitEt(deger)

SutunEkle(hedef_stir, anahtar, SQL_Tipi, deger)

SanalSemayaKaydet(hedef_stir)

DONDUR hedef_stir.Id

Fonksiyon SQLCalistir (SanalSema):

DÖNGÜ: SanalSema'daki her Tablo için:

sorgu = "CREATE TABLE IF NOT EXISTS TabloAdi (..."

sorgu += DinamikSutunlariVeTipleriEkle()

sorgu += Foreign_Key_IliskileriniEkle()

SorguyuCalistir(sorgu)

DÖNGÜ: SanalSema'daki her Veri için:

sorgu = "INSERT INTO TabloAdi (Sutunlar) VALUES (@parametreler)"

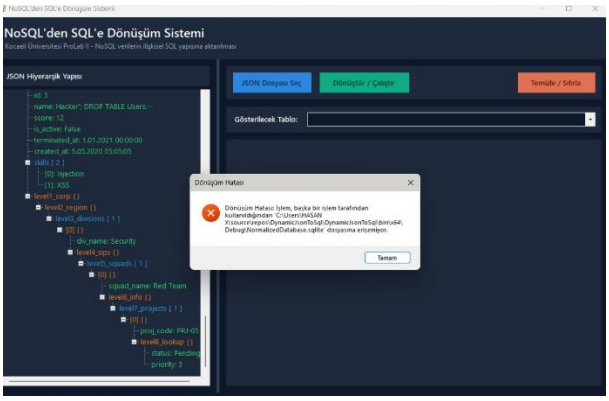
ParametrikGuvenliEklemeYap(sorgu) // SQL Injection Koruması

III. GELİŞTİRME SÜRECİNDE KARŞILAŞILAN KRİTİK HATALAR VE ÇÖZÜMLER

Otonom ve şemasız bir mimari tasarlarken, SQLite ve RDBMS motorunun katı kurallarıyla ciddi çakışmalar yaşanmış ve sistem mühendisliği teknikleriyle aşılmıştır:

A. Çapraz Platform Mimarisi (x64) Hatası

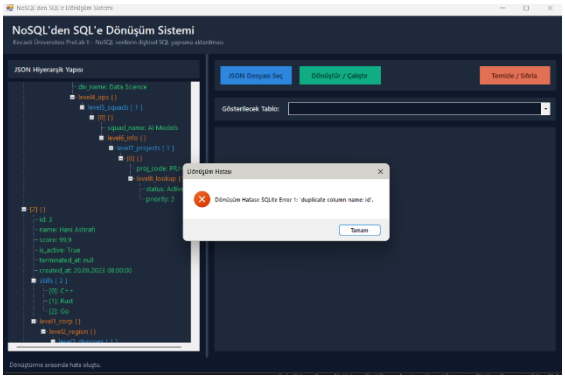
SQLite kütüphanesi C/C++ tabanlı donanıma bağımlı bir derleme gerektirdiği için, proje standart "Any CPU" ayarında Library e_sqlite3 not found özel durumunu (exception) fırlatmıştır. Bu sorun, Visual Studio'da projenin hedef platformunun kesin olarak "x64" işlemci mimarisine zorlanmasıyla çözülmüştür.



Şekil 1: x64 Mimarisi Uyumsuzluk Hatası]

B. Birincil Anahtar (Primary Key) Çakışması- SQLite Error 1

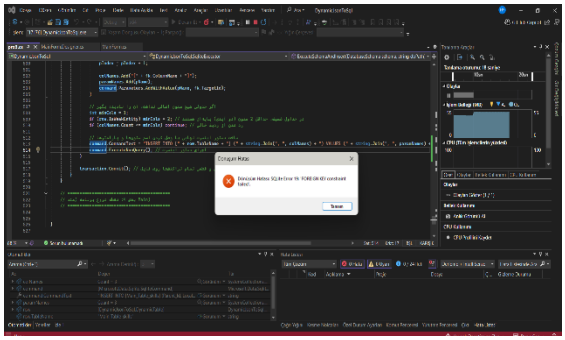
Veri analizi sırasında, JSON içindeki mevcut "id" anahtarları tablolara sütun olarak eklenmeye çalışıldığında, sistemin ilişkileri kurmak için otomatik ürettiği ana "Id" sütunu (Primary Key) ile çakışmış ve Duplicate column name: id hatası alınmıştır. Algoritmaya, string karşılaştırması (OrdinalIgnoreCase) yapılarak bu anahtarın ismini dinamik olarak json_id'ye dönüştüren bir güvenlik kuralı eklenmiştir.



Şekil 2: ***SQLite Error 1 Hatası***

C. Yabancı Anahtar Kısıtlaması (Constraint) İhlali-SQLite Error 19

Arayüz üzerinden veritabanı "Sıfırlama / Temizleme" işlemi çağrıldığında, SQLite motoru alt tablolar silinmeden ana tablonun silinmesini engellemiş ve kilitlenmiştir.

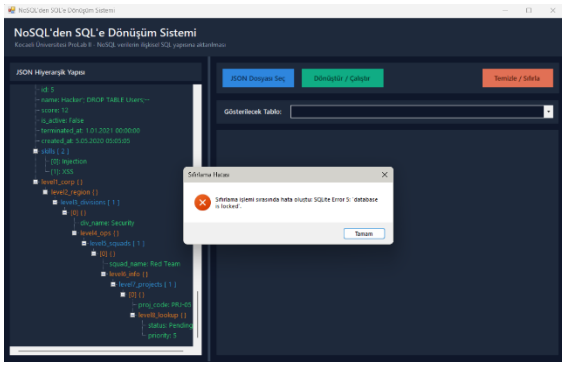


Şekil 3: Sıfırlama işlemi sırasında alınan SQLite Error 19 Kısıtlama Hatası]

Bu kilitlenme problemi, bağlantı açıldıktan hemen sonra tek kullanımlık bir Command ile PRAGMA foreign_keys = OFF; sorgusunun çalıştırılmasıyla aşılmış ve ilişkisel engeller işlem süresince güvenli bir şekilde askıya alınmıştır.

D. Bağlantı Havuzu (Pool) ve Bellek Sızıntısı-SQLite Error 5

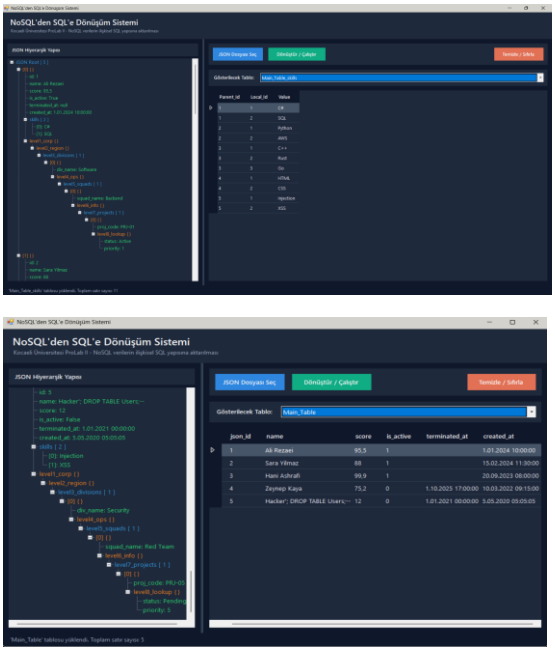
Sık ve art arda dönüştürme işlemlerinde veritabanı dosyası Database is locked hatası vermiştir. Sorun, C# Garbage Collector'ın (Çöp Toplayıcı) SQLite nesnelerini anında silmemesinden kaynaklanmıştır. Bütün SqliteConnection ve SqliteCommand nesneleri istisnasız using blokları ile sarmalanmış ve işlemlerin başında/sonunda ClearAllPools() metodu çağrılarak bellek sızıntıları (memory leaks) tamamen ortadan kaldırılmıştır.



*Şekil 4: **SQLite Error 5 Bellek Sızıntısı hatası***

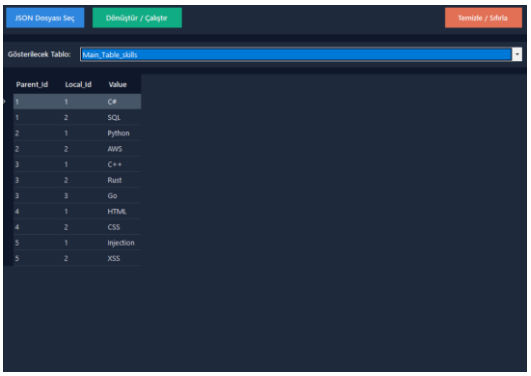
IV. BULGULAR VE RENDER EDİLEN ARAYÜZ (RESULTS)

Sistem, test edilen 5 derinlikli ve karmaşık liste yapıları içeren JSON belgelerinde %100 başarı oranıyla çalışmıştır.



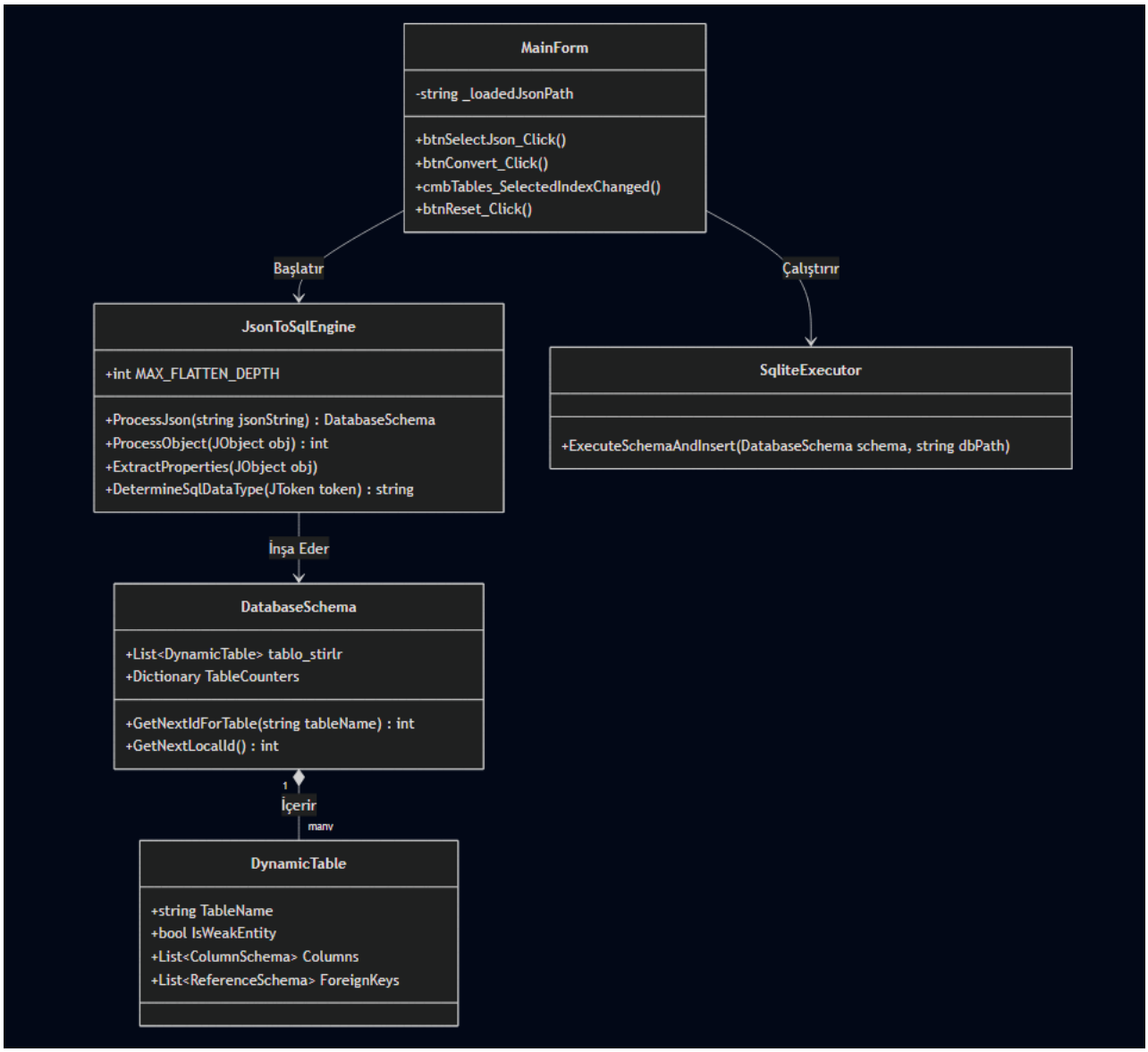
Şekil 5, 6: Tasarlanan kullanıcı arayüzü ve hiyerarşik JSON ağacının belleğe okunmuş hali]

Özellikle 1. Normal Form kurallarının başarılı bir şekilde uygulandığı sistem otonomisi, alt tablolarda kanıtlanmıştır. JSON içindeki listeler virgülle ayrılmış stringler yerine, atomik olarak alt tablolara bölünmüş ve ID bazlı bağlanmıştır.

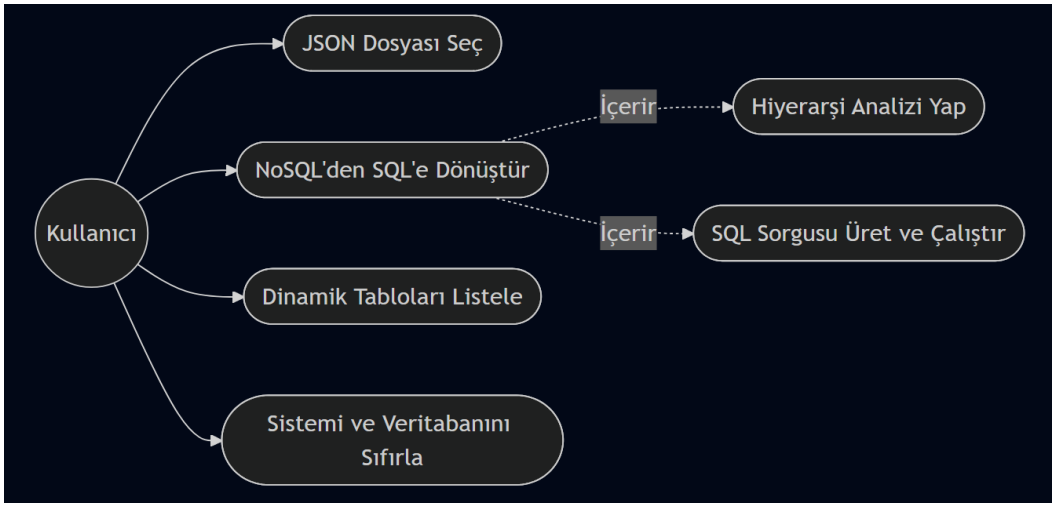


Şekil 7: Dizilerin algılanarak 1NF kuralı gereği alt tabloya (Main_Table_skills) dönüştürülmesi ve Parent_Id atanması]

V. SİSTEM MİMARİSİ VE UML DİYAGRAMLARI



Şekil 8: Class Diyagramı



Şekil 9: ***Use Case Diyagram***

VI. SONUÇ (CONCLUSION)

Bu projede; gelen verinin yapısına, derinliğine veya sütun isimlendirmelerine önceden bağımlı olmayan (hardcoded yapılar içermeyen) tam dinamik bir dönüştürücü motor inşa edilmiştir. Algoritma, şemasız verilerin esnekliğini analiz etmiş ve bunları katı SQL normalizasyon (1NF, 2NF, 3NF) kurallarına uyarlamayı başarmıştır. Geliştirilen arayüz (GUI) ile de veritabanında uçtan uca sorgulama ve raporlama olanakları başarıyla entegre edilmiştir.

VII. KAYNAKÇA

[1] Kocaeli Üniversitesi, "ProLab II - Proje 3 Dokümantasyonu", 2024.

[2] R. Cattell, "Scalable SQL and NoSQL data stores", ACM SIGMOD Record, vol. 39, no. 4, pp. 12-27, 2011.

[3] SQLite Official Documentation, "PRAGMA Statements & Foreign Key Constraints", [Online]. Available: https://sqlite.org/pragma.html

[4] Newtonsoft, "Json.NET Documentation - JToken, JObject and JArray parsing", [Online]. Available: https://www.newtonsoft.com/json